

**Instituto Infnet**

**Desenvolvimento Seguro de Aplicações Web TP2**

**Aluno: Breno Lobato Vila Nova**

**Curso: Engenharia de software**

**Cidade: Rio de Janeiro**

**Escola: ESTI**

**Data: 02/09/2026**

**Professor: Fabiano de Moraes Domingues**

### **Exercício 1**

#### **Contexto**

Antes de qualquer modelagem formal de ameaças, o time de segurança da empresa pede que cada squad pense como um atacante pensaria: quais ações mal-intencionadas alguém tentaria contra o eventos-api? Essa prática, chamada de misuse cases, precede o threat modeling formal e ajuda a equipe a enxergar o sistema pela perspectiva de quem quer explorá-lo, e não apenas pela perspectiva de quem quer que ele funcione. Em uma retrospectiva recente, o time identificou que pular essa etapa em outro projeto levou a um threat model incompleto, que só cobria ameaças óbvias e deixou de fora vetores relacionados a dados de terceiros. Você foi designado para produzir a primeira rodada de misuse cases do eventos-api, cobrindo os principais vetores de ataque relevantes para uma API de gestão de eventos e inscrições, antes de a equipe avançar para a categorização formal com STRIDE.

#### **Tarefa**

1. Elabore pelo menos quatro misuse cases para o eventos-api, cada um descrevendo um ator malicioso, uma ação indesejada e o impacto potencial.
2. Relacione cada misuse case a um vetor de ataque concreto observável na aplicação construída no TP1 (por exemplo, ausência de autenticação nas rotas de criação de evento).
3. Priorize os misuse cases listados por impacto, justificando a ordem escolhida.

## Resposta

### Tarefa 1. & .2

| Misuse Case | Ator             | Ação Indesejada                                | Impacto Potencial                                                      | Vetor de Ataque                                                 |
|-------------|------------------|------------------------------------------------|------------------------------------------------------------------------|-----------------------------------------------------------------|
| 1           | Atacante Externo | Criar evento se passando por outro organizador | Criação de dados fraudulentos                                          | POST /events não necessita de login                             |
| 2           | Atacante Externo | Consultar eventos privados de organizadores    | Exposição de informações desses eventos privados                       | GET /events não checa permissão de acesso                       |
| 3           | Atacante Externo | Envio excessivo de requisições                 | Consumo alto de recursos computacionais e potencial negação de serviço | Rotas sem login necessário e ausência de proteção contra abusos |
| 4           | Atacante Externo | Enviar dados maliciosos nos campos de eventos  | Inserção de dados maliciosos no banco                                  | Não existe validação ou sanitização dos dados de entrada        |

### Tarefa 3.

- Os casos foram priorizados com base no impacto potencial sobre os dados e a disponibilidade do sistema:
  1. Criação não autorizada
  2. Acesso não autorizado
  3. Envio de dados maliciosos
  4. Envio excessivo de requisições

## Exercício 2

### Contexto

Com os misuse cases levantados, o próximo passo pedido pelo tech lead é categorizar as ameaças de forma sistemática usando o framework STRIDE, padrão adotado pela empresa para toda modelagem de ameaças de novos serviços antes de qualquer deploy em produção. A ideia é que cada componente da API (autenticação, rotas de eventos, banco de dados simulado) seja avaliado

individualmente contra as seis categorias do STRIDE, para que nenhuma classe de ameaça relevante seja esquecida na análise, já que uma análise informal anterior, feita sem framework, deixou passar uma ameaça de elevação de privilégio que só foi descoberta em produção.

### Tarefa

1. Aplique o framework STRIDE a pelo menos três componentes do eventos-api (por exemplo, rota de criação de evento, camada de autenticação futura, armazenamento de dados).
2. Para cada componente, identifique pelo menos uma ameaça correspondente a uma das seis categorias STRIDE (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege).
3. Registre os resultados em uma tabela com as colunas Componente, Categoria STRIDE, Ameaça identificada.

### Resposta

#### Tarefa 1. & 2. & 3.

| Componente                   | Categoria STRIDE       | Ameaça Identificada                                                                                                     |
|------------------------------|------------------------|-------------------------------------------------------------------------------------------------------------------------|
| EventsRouter#create_handler  | Spoofing               | Um atacante pode criar um evento sem autenticação, se passando por outro organizador.                                   |
| EventsRouter#get_all_handler | Information Disclosure | Um atacante pode consultar eventos sem checagem de permissão de acesso, obtendo informações que deveriam ser privadas.  |
| EventsRepository#save        | Denial of Service      | O armazenamento em memória pode consumir recursos além do esperado caso sejam enviados muitos eventos para a aplicação. |

### Exercício 3

#### Contexto

O time de segurança quer consolidar o trabalho dos dois exercícios anteriores em um artefato único e formal: o threat model do eventos-api. Esse documento será usado como referência oficial nas próximas etapas da disciplina e também pelo

time de operações para priorizar correções antes de qualquer exposição do serviço a usuários externos. Um threat model incompleto, sem mapeamento claro de ativos e superfícies de ataque, foi rejeitado recentemente em outro projeto da empresa por não permitir rastrear quais mitigações cobrem quais ameaças, o que obrigou a equipe a refazer o documento inteiro sob pressão de prazo. Você precisa evitar esse retrabalho entregando um artefato completo desde a primeira versão.

### **Tarefa**

1. Construa o threat model do eventos-api, mapeando os ativos principais (dados de usuários, dados de eventos, credenciais futuras), as superfícies de ataque identificadas e as mitigações propostas para cada ameaça do Exercício 2.
2. Garanta que cada ameaça do STRIDE mapeado no Exercício 2 apareça no threat model com pelo menos uma mitigação associada.
3. Apresente o threat model em um documento único e legível, adequado para ser consultado por outro desenvolvedor sem contexto prévio.

### **Resposta**

#### **Tarefa 1. & 2. & 3.**

- pdf: <https://github.com/kick250/infnet-security-development/tree/main/docs/tp2/exercise3/threat-model.pdf>
- md: <https://github.com/kick250/infnet-security-development/tree/main/docs/tp2/exercise3/threat-model.md>

### **Exercício 4**

#### **Contexto**

Um arquiteto sênior revisando o eventos-api apontou que a modelagem de ameaças feita até aqui carece de uma visão formal da arquitetura do sistema: quais são os componentes reais, onde ficam as fronteiras de confiança entre eles e como os dados fluem de um componente a outro. Segundo ele, threat models construídos sem essa visão de partições tendem a ficar genéricos demais, cobrindo ameaças óbvias e deixando passar riscos específicos das fronteiras reais do sistema. Ele pediu que a equipe formalize essa visão, complementando o threat model já construído, antes de avançar para a implementação de autenticação na próxima etapa da disciplina.

### **Tarefa**

1. Identifique as fronteiras de segurança do eventos-api, particionando o sistema em pelo menos três componentes distintos (por exemplo, cliente, camada de API, armazenamento de dados).

2. Mapeie o fluxo de dados entre essas partições, indicando explicitamente onde uma requisição cruza uma fronteira de confiança.
3. Relacione, em um parágrafo curto, como essa visão de partições confirma ou refina as ameaças já identificadas no threat model do Exercício 3.

## Resposta

### Tarefa 1.

#### Partições

1. **Cliente:** Navegador ou ferramenta HTTP que envia as requisições.
2. **API:** Aplicação FastAPI responsável por receber e processar as requisições.
3. **Armazenamento:** Estrutura em memória usada para armazenar os dados da API.

### Tarefa 2.

#### Fluxo de dados

- Cliente => trust boundary > API => Armazenamento

A requisição **passa pelo trust boundary ao entrar na API**, pois os dados passam de um ambiente não confiável para o nosso sistema.

### Tarefa 3.

- Podemos ver pelo fluxo que a API processa a requisição e acessa o armazenamento para consultar ou salvar os dados de eventos.

## Exercício 5

### Contexto

A empresa está avaliando abrir parte do eventos-api para parceiros externos consumirem via integração automatizada, além dos usuários finais que acessam pelo navegador. Antes de decidir o modelo de autenticação, o tech lead pede uma análise dos vetores de ataque possíveis segundo os três eixos de segurança de APIs (design, implementação, infraestrutura), para garantir que a decisão de arquitetura considere riscos além da simples validação de senha. Em uma discussão recente com o time de infraestrutura, ficou claro que decisões de autenticação tomadas sem olhar para o eixo de infraestrutura já causaram exposição indevida de endpoints internos em outro serviço da empresa, então essa análise precisa ser levada a sério antes da implementação.

### Tarefa

1. Para cada um dos três eixos de segurança de APIs (design, implementação, infraestrutura), identifique pelo menos um vetor de ataque relevante para o cenário de abertura do eventos-api a parceiros externos.
2. Indique, para cada vetor identificado, se ele já está coberto pelo threat model construído anteriormente ou se representa uma lacuna nova.
3. Justifique por que considerar os três eixos, e não apenas o eixo de implementação, é necessário antes de expor a API a consumidores externos.

### Resposta

#### Tarefa 1. & 2.

| Eixo           | Vetor de ataque                                                                               | Coberto?    |
|----------------|-----------------------------------------------------------------------------------------------|-------------|
| Design         | Expor integração sem uma limitação baseada nos recursos que podem ser acessados pelo parceiro | Lacuna Nova |
| Implementação  | Ausência de autenticação nas rotas                                                            | Coberto     |
| Infraestrutura | Expor recursos internos acessados pela nossa API                                              | Lacuna Nova |

#### Tarefa 3.

- Considerar esses três eixos é necessário porque a segurança da API não depende apenas da implementação. O design define quais recursos são disponibilizados, a implementação garante acesso autorizado e a infraestrutura protege a API e seus recursos internos contra acessos indevidos.

## Exercício 6

### Contexto

Com a modelagem de ameaças consolidada, chegou a hora de implementar a primeira camada real de autenticação do eventos-api. O produto exige que apenas usuários cadastrados possam criar e gerenciar eventos, e que senhas nunca sejam armazenadas em texto plano, requisito não negociável levantado pelo time de segurança na revisão do threat model, depois de um incidente em outro projeto da empresa em que senhas ficaram expostas em um vazamento de banco de dados por estarem salvas sem hashing. Além disso, rotas de edição de evento precisam garantir que apenas o organizador dono do evento possa modificá-lo, fechando a lacuna de autorização que o threat model já havia identificado.

### Tarefa

1. Implemente autenticação com OAuth2PasswordBearer no eventos-api, aplicando hashing de senha com bcrypt via passlib.
2. Proteja a rota de edição de evento usando dependency injection do FastAPI, garantindo que apenas o organizador dono do evento (verificação de ownership) possa executá-la.
3. Demonstre, com um teste manual via httpie ou similar, o comportamento da rota de edição para um usuário autenticado dono do evento e para um usuário autenticado que não é dono.

### Resposta

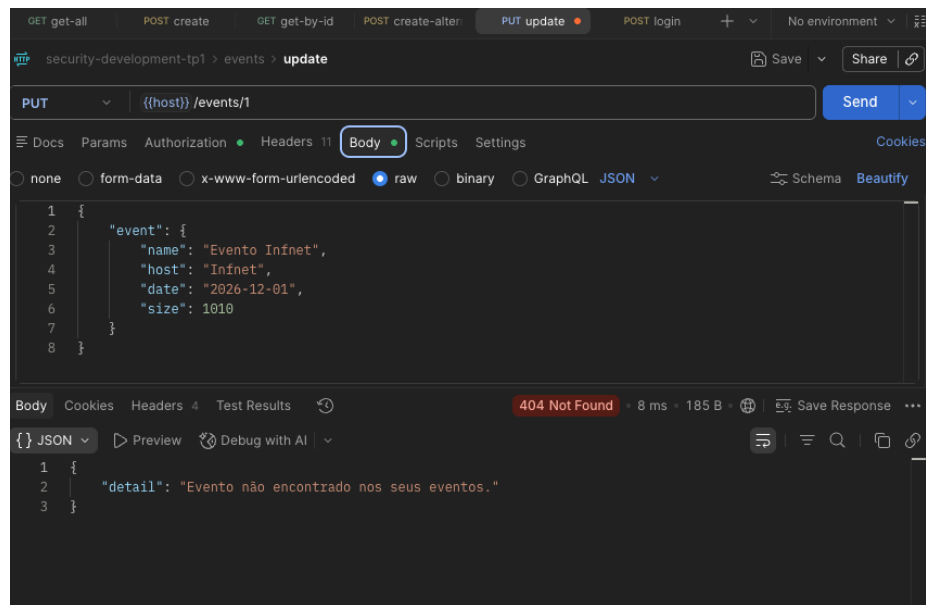
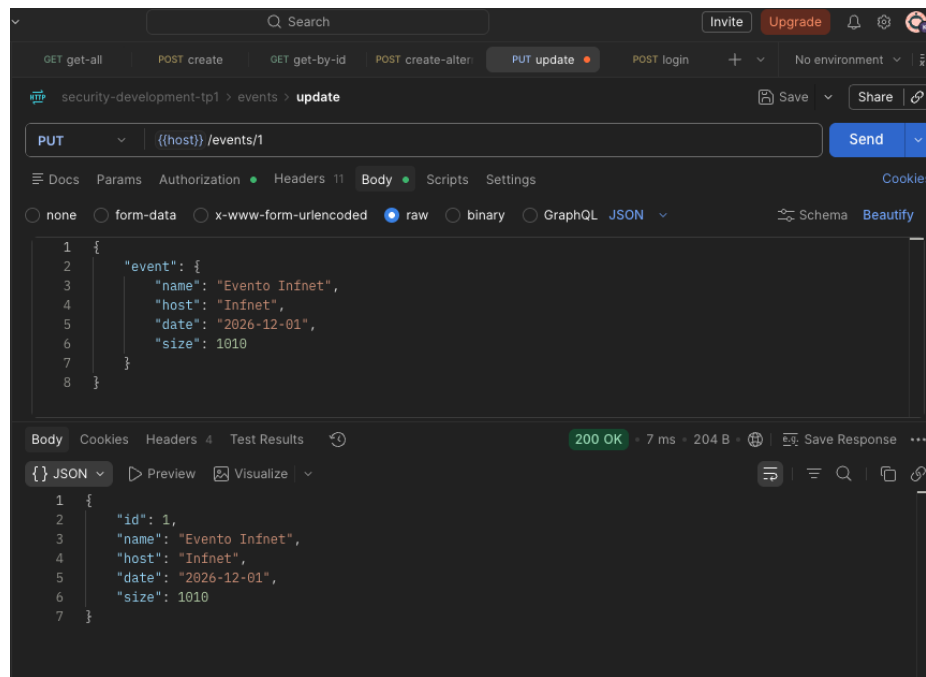
#### Tarefa 1.

- autenticação: /app/auth.py
- hashing bcrypt: /app/repositories/users\_repository.py

#### Tarefa 2.

- dependency injection:
  - arquivo: /app/routes/events\_router.py
  - metodo: `EventsRouter#update_handler`
- verificação de ownership:
  - arquivo: /app/routes/events\_router.py
  - metodo: `EventsRouter#__check_ownership`

#### Tarefa 3.



## Exercício 7

### Contexto

O time de compliance exige que sessões de usuário sigam boas práticas de segurança, incluindo expiração de token e suporte a múltiplos fatores de autenticação em contas administrativas, já que contas de administrador têm acesso a dados de todos os organizadores e participantes da plataforma. Ao mesmo tempo, o tech lead está decidindo entre RBAC e ABAC para controlar quem pode acessar quais recursos do eventos-api, já que o sistema terá papéis distintos (organizador, participante, administrador) com regras de acesso que podem variar por contexto, e uma escolha malfeita agora seria custosa de reverter depois que a base de usuários crescer.

### Tarefa

1. Implemente JWT com OAuth2 no eventos-api, incluindo expiração de token e um fluxo simulado de segundo fator de autenticação (MFA) para o papel de administrador.
2. Compare RBAC, ABAC e autorização por recurso, explicando qual modelo você recomendaria para o eventos-api e por quê, considerando os papéis organizador, participante e administrador.
3. Documente a decisão de modelo de autorização em um parágrafo técnico, citando pelo menos um cenário concreto do domínio de eventos em que o modelo escolhido se aplica.

### Resposta

#### Tarefa 1.

- OAuth2 com JWT: `/app/routes/authentication_router.py`

#### Tarefa 2.

- **RBAC:** Controla o acesso com base no papel do usuário, como administrador, participante ou organizador.
- **ABAC:** Controla o acesso com base em atributos do usuário, recurso e contexto da requisição.
- **Autorização por recurso:** Verifica se o usuário possui permissão sobre um recurso específico, como evento ou inscrição.

Eu recomendaria **RBAC combinado com autorização por recurso**, pois o RBAC facilita o controle dos papéis, como organizador, participante e administrador, enquanto a autorização por recurso permite verificar se o usuário possui acesso ao evento específico.

#### Tarefa 3.

- O modelo de RBAC com autorização por recurso permitiria que um organizador pudesse editar apenas os eventos que pertencem a ele, enquanto um participante poderia apenas ver ou se inscrever em eventos permitidos.

## Exercício 8

### Contexto

A empresa parceira que vai integrar via API pediu suporte a comunicação máquina-a-máquina (M2M), sem intervenção de um usuário humano no fluxo de autenticação, além do fluxo já existente para usuários finais autenticados pelo navegador. O contrato comercial assinado com o parceiro especifica exatamente quais operações ele pode executar, e o time jurídico da empresa exige que essa limitação seja tecnicamente garantida, não apenas descrita em papel. O time de segurança pede que os escopos de acesso sejam explícitos, para que o parceiro externo só consiga executar exatamente as operações autorizadas em contrato, nada além disso, mesmo que o token seja comprometido.

### Tarefa

1. Selecione o fluxo OAuth 2.0 mais adequado para o cenário M2M do parceiro externo, justificando a escolha frente ao fluxo já usado para usuários finais.
2. Configure escopos OAuth e claims JWT específicos para RBAC, distinguindo o escopo de um usuário organizador do escopo do parceiro M2M.
3. Demonstre, com um exemplo de payload de token decodificado, quais claims e escopos diferenciam os dois tipos de cliente.

### Resposta

#### Tarefa 1.

- Para esse caso do M2M, o fluxo que eu escolhi foi o **Client Credentials**, pois o parceiro irá se comunicar máquina-a-máquina com a API. Diferente do fluxo de usuários normais, o parceiro também deseja acesso somente a determinados escopos de recursos da API.

#### Tarefa 2.

- escopos OAuth e claims JWT: `/app/services/token_service.py`

#### Tarefa 3.

- Na implementação feita as permissões são representadas no JWT através dos claims `access_type` e `allowed_resources`. Sendo a função de cada uma delas:
  - `access_type`: Identificar o tipo de acesso do cliente. Podendo ser `standard` para usuários normais ou `by_resources` para parceiros como o M2M.

- `allowed_resources`: Define quais recursos e ações podem ser acessados.

Exemplo de payload JWT para um usuário organizador:

```
{
  "sub": "1010",
  "exp": 1788401989,
  "access_type": "standard",
  "allowed_resources": {},
  "created_at": 1788400189.957734
}
```

Exemplo de payload JWT para um parceiro máquina-a-máquina (M2M):

```
{
  "sub": "1012",
  "exp": 1788401989,
  "access_type": "by_resources",
  "allowed_resources": {
    "events": {
      "read": true,
      "write": true,
      "delete": false
    }
  },
  "created_at": 1788400189.957734
}
```